Chris Garlick

INTERVIEW PREPARATION

Workhorse

Software Engineer – Automation & Operations. PHP, MySQL, AWS and AI-assisted helpdesk automation.

PREPARED FOR	PREPARED BY	DATE
Workhorse interview, 23 October	Chris Garlick	22 October 2026

Contents

01	What they're hiring for	2
02	What they want, what to show	3
03	From Jira ticket to production	4
04	Reviewing code you didn't write	5
05	How a RAG pipeline works	7
06	Drafting, resolution and escalation	8
07	SQL you should be fluent in	10
08	Performance and safe schema changes	11
09	CI/CD and reliable releases	12
10	AWS monitoring and alerting	14
11	Typeset: memory on a small server	15
12	PHP refresher	16
13	Stories to have ready	18
14	Questions they're likely to ask	19
15	Questions to ask them	20
16	Tonight and tomorrow	21

01

THE ROLE

What they’re hiring for

A broad, practical engineering role on a PHP / MySQL / AWS platform: ship Jira tickets with AI-assisted tools, keep the helpdesk moving with automation, and keep the platform healthy.

TITLE	Software Engineer – Automation & Operations
STACK	PHP, MySQL, AWS
DAY TO DAY	Jira tickets, scoped, built, reviewed, tested and deployed with their existing AI-assisted workflow
THE BIG PROJECT	Their AI-powered RAG pipeline for the helpdesk: better ticket enrichment, then automated drafting, resolution and escalation
ALSO	SQL and migrations, CI/CD, AWS monitoring and alerting, implementation support, the outsourced infrastructure team
LEVEL	Mid-level. Strong SQL, sound fundamentals, deployment, automation and cloud experience
NOTE	PHP is *beneficial but not essential*, so your PHP is a clear advantage

Read between the lines

Improving existing systems rather than always building from scratch and **confident in assessing the quality and reliability of AI-generated outputs** are the two phrases to land. They already have the AI tools; they want someone who makes them *reliable*, not someone who just generates more code.

02

THE ROLE, MAPPED

What they want, what to show

Every responsibility in the job description, with the evidence to bring. Fill the gaps tonight.

They want	What to show
Deliver from Jira tickets	How you take a vague ticket to a scoped, tested, deployed change.
AI-assisted development	How you use Claude Code day to day, and how you review what it writes.
Helpdesk support and RAG	An LLM or retrieval feature you built; how you'd measure and improve it.
Strong SQL	A query you optimised, a schema you designed, a migration you ran safely.
CI/CD and automation	A pipeline you built; a repetitive task you automated away.
AWS monitoring	Metrics and alarms you've set up, and an incident you caught or diagnosed.
Investigation	A production bug you tracked down: how you found it, how you fixed it.
Working with others	Helping non-developers (helpdesk, implementation) and outside specialists.

Your strongest stories

Typeset: a render API you built and run in production, which hit memory limits on a small server; you diagnosed the OOMs, added swap, and know concurrency and timeouts are next. That is *monitoring, investigation and operations* in one story. **Claude Code**: you build with coding agents every day and review their output. **PHP / Laravel**: years of it.

03

SOFTWARE DEVELOPMENT

From Jira ticket to production

They want ownership: you help scope the ticket, not just pick it up.

> **Why it matters**
> **Take ownership of Jira development tickets, help scope technical requirements and deliver** is the first responsibility listed. Show a repeatable process.

Scope

1 **Understand the problem**

Who reported it, what they expected, how to reproduce it. Ask before you build.

2 **Find the code**

Trace the path through the codebase; note what else touches it.

3 **Write acceptance criteria**

What done looks like, edge cases, and what's out of scope. Add it to the ticket.

Build

4 **Plan with the agent**

Give the AI tool the ticket, the relevant files and the constraints; agree a plan first.

5 **Small, focused changes**

One concern per PR; no drive-by refactors in a bug fix.

6 **Test**

A failing test that reproduces the bug, then the fix. Cover the edge cases from scoping.

Ship

7 **Review**

Read every line yourself before anyone else does, then a peer review.

8 **Deploy**

Through the pipeline, with a way to roll back; database changes go first and stay backward compatible.

9 **Close the loop**

Check it in production, update the ticket, tell the person who reported it.

> **Say this**
> *"I treat the ticket as the start of a conversation. I'll reproduce it, write down what done looks like, then build it in small reviewed steps, and I don't close it until I've seen it working in production."*

04

AI-ASSISTED DEVELOPMENT

Reviewing code you didn't write

They already use AI tools. The skill they're hiring for is judging the output.

> **Why it matters**
> **Reviewing generated code to ensure changes are well-targeted, maintainable, secure and properly tested.** Those four words are their checklist; use them back.

Their word	What you check
Well-targeted	Does the diff only touch what the ticket needs? Agents love to "improve" nearby code. Revert anything unrelated.

Their word	What you check
Maintainable	Does it match the codebase's patterns and naming? No new abstractions for one use, no duplicated helpers that already exist.
Secure	Parameterised SQL, escaped output, authorisation checks on every route, no secrets in code or logs, input validated.
Properly tested	Tests that would fail without the change. Watch for tests that only test mocks, or assertions weakened to pass.

Where AI output goes wrong

- **Confident invention**: a function, column or config option that doesn't exist. Check it does.
- **Silent scope creep**: a 40-line fix becomes a 400-line diff.
- **Happy path only**: nulls, empty lists, timezones and concurrency get missed.
- **Swallowed errors**: `try/catch` that hides the real failure.
- **Tests bent to fit**: the test changed to match buggy behaviour, not the other way round.

Improving the workflow itself

- Project instructions (e.g. `CLAUDE.md`) with conventions, commands and "don't touch" areas.
- Make the agent run the tests and linters before it says it's done.
- Reusable prompts or skills for common tickets: a bug fix, a migration, a new endpoint.
- Automated review on every PR as a first pass, never a replacement for a human.

> **Say this**
> *"I use Claude Code every day, and the more I use it the more I see my job as setting it up well and reviewing hard. I read every diff as if a new colleague wrote it."*

05

HELPDESK AUTOMATION

How a RAG pipeline works

Retrieval-augmented generation: find the relevant information first, then let the model answer from it.

> **Why it matters**
> Maintaining and developing **their existing AI-powered RAG pipeline** is the most distinctive part of the job. Expect detailed questions here.

Indexing (ahead of time)

1 **Collect sources**

Help articles, past resolved tickets, product docs, release notes, known issues.

2 **Chunk**

Split into passages of a few hundred tokens, by heading or section, with a little overlap.

3 **Embed and store**

Turn each chunk into a vector; store it with metadata (product, version, date, source URL).

Answering (per ticket)

4 **Build the query**

From the ticket's subject and body, plus customer context: plan, product, version.

5 **Retrieve**

Top matches by vector similarity, ideally combined with keyword search (hybrid), filtered by metadata.

6 **Rerank**

Reorder the candidates with a reranker so the best few go to the model.

7

Generate

The model drafts an answer *only* from the retrieved context, with citations.

Problem	Fix
Finds the wrong articles	Better chunking, hybrid search for error codes and product names, metadata filters, reranking.
Out-of-date answers	Re-index on change; store dates; prefer newer sources; retire old articles.
Makes things up	Answer only from context, require citations, allow “I don’t know”, lower temperature.
Can’t tell if it’s improving	An evaluation set of real past tickets with known good answers; measure every change.

06

HELPDESK AUTOMATION

Drafting, resolution and escalation

The job description says *progressively*. Automate in stages, with a human in the loop until the numbers say otherwise.

Stage 1: Enrich

1

Add context to every ticket

Customer’s account, recent errors from logs, related articles and similar past tickets, for the agent to see.

2

Classify

Category, product area, urgency and sentiment, so tickets reach the right person.

Stage 2: Draft

3

Suggest a reply

The helpdesk agent edits and sends. Nothing goes to the customer unreviewed.

4

Measure

How often drafts are sent unchanged, lightly edited or discarded. That’s your quality signal.

Stage 3: Resolve

5 **Auto-send for narrow, safe categories**

Password resets, how-to questions with a clear article, only above a confidence threshold.

6 **Always offer a human**

Every automated reply says how to reach a person.

Escalate

7 **Clear rules**

Low confidence, angry customer, billing or data issues, security, repeated contact, or a likely bug go to a person.

8 **Hand over with context**

The escalation carries the summary, what was tried and the retrieved sources.

Risks to raise before they do

Prompt injection: ticket text is untrusted input; it must never be able to change the model's instructions or trigger actions. **Personal data**: what goes to the model provider, and is it logged? **Wrong answers at scale**: one bad article can now reach hundreds of customers, so monitor and keep a kill switch.

Say this

"I'd start by measuring what the pipeline does today against real past tickets, because you can't improve retrieval you can't measure. Then automate the narrow, safe cases first and widen it as the draft-acceptance rate proves it."

07

DATABASE MANAGEMENT

SQL you should be fluent in

Strong SQL is the first thing in the candidate profile. Expect a live query or a whiteboard question.

Why it matters

Write and maintain SQL, manage database updates, schema changes and migrations, and ensure database integrity and performance. Fluency here is non-negotiable.

Topic	What to say
Joins	INNER vs LEFT; LEFT JOIN … WHERE b.id IS NULL to find missing rows.
Aggregation	GROUP BY with HAVING to filter groups; COUNT(DISTINCT …).
Window functions	ROW_NUMBER, RANK, LAG, running totals; MySQL 8 has them.
CTEs	WITH … AS for readable multi-step queries; recursive CTEs for trees.
Indexes	Composite indexes follow the leftmost-prefix rule; cover the WHERE then the ORDER BY.
EXPLAIN	Look for type = ALL (full scan), “Using filesort”, “Using temporary”, rows examined.
Transactions	InnoDB; BEGIN / COMMIT; default isolation REPEATABLE READ; keep them short.
Integrity	Foreign keys, NOT NULL, unique constraints, correct types (DECIMAL for money).

```sql
-- Latest ticket per customer, with how many they've raised
WITH ranked AS (
    SELECT t.*,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER
BY created_at DESC) AS rn,
           COUNT(*)     OVER (PARTITION BY customer_id)
AS total
    FROM tickets t
    WHERE created_at ⩾ NOW() - INTERVAL 30 DAY
)
SELECT customer_id, id AS latest_ticket, subject, total
FROM ranked
WHERE rn = 1 AND total ⩾ 3
ORDER BY total DESC;
```

08

DATABASE MANAGEMENT

Performance and safe schema changes

How to find a slow query, and how to change a big table without taking the site down.

Finding a slow query

Diagnose

1

Find it

Slow query log, Performance Insights on RDS, or performance_schema.

2

Read the plan

EXPLAIN, or EXPLAIN ANALYZE (MySQL 8.0.18+) for actual timings.

3

Fix the cause

Add or reorder an index, avoid functions on indexed columns, fix N+1 queries in the app, paginate.

```
EXPLAIN ANALYZE
SELECT id, subject FROM tickets
WHERE status = 'open' AND assigned_to = 42
ORDER BY created_at DESC LIMIT 20;

-- Serves the filter and the sort in one index
CREATE INDEX idx_tickets_status_assignee_created
    ON tickets (status, assigned_to, created_at);

-- Not this: a function on the column can't use the index
-- WHERE DATE(created_at) = '2026-10-23'
-- Do this instead
-- WHERE created_at ⩾ '2026-10-23' AND created_at <
'2026-10-24'
```

Changing a big table safely

EXPAND, THEN CONTRACT

Add the new column, deploy code that writes both, backfill in batches, switch reads, then drop the old column in a later release.

ONLINE DDL	MySQL 8 can add many columns with ALGORITHM=INSTANT, and many index changes with INPLACE, without locking writes.
BIG TABLES	pt-online-schema-change or gh-ost copy the table in the background and swap it in.
BACKFILLS	In batches of a few thousand rows, with a pause between, so you don't flood replication.
ALWAYS	A backup or snapshot first, tested on a copy of production data, and a rollback plan.

09

DEPLOYMENT AND AUTOMATION

CI/CD and reliable releases

Maintain and improve their pipelines, and automate the repetitive work.

Why it matters

Maintain and improve existing CI/CD pipelines, automate repetitive development tasks and support reliable software releases. Note *existing*: understand it before you change it.

A good pipeline

1 **On every push**

Install dependencies (cached), lint and static analysis (PHPStan), run the tests.

2 **On merge**

Build one artefact, deploy it to staging, run smoke tests.

3 **To production**

The same artefact, with migrations first, then a health check and an easy rollback.

```yaml
# GitHub Actions: the shape of a PHP pipeline
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      mysql: { image: mysql:8.0, env: { MYSQL_ROOT_PASSWORD:
root, MYSQL_DATABASE: test } }
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.3' }
      - run: composer install --no-interaction --prefer-dist
      - run: vendor/bin/phpstan analyse
      - run: vendor/bin/phpunit
```

Release practice	Why
Small, frequent deploys	Less to go wrong; easier to find what did.
Backward-compatible migrations	Old and new code both work during the deploy.
Feature flags	Ship code dark, turn it on for some customers first.
Automatic rollback on failed health check	Bad deploys fix themselves in minutes.

Automating repetitive tasks

Good examples to offer: generating release notes from merged PRs, setting up a local environment with one command, refreshing a staging database with anonymised data, and AI review of every PR as a first pass.

10

INFRASTRUCTURE MONITORING

AWS monitoring and alerting

Availability, memory, CPU and performance are named explicitly. Know which need extra setup.

Why it matters

Implement and maintain AWS monitoring and alerting, covering system availability, memory utilisation, CPU capacity, performance and potential issues. This is a gap they want filled.

The detail that shows you've done it

EC2 sends **CPU** to CloudWatch by default, but **not memory or disk space**. Those need the **CloudWatch agent** installed on the instance. RDS reports memory as **FreeableMemory** without it.

Watch	Metric	Alert when
Availability	ALB HealthyHostCount; a Synthetics canary or uptime check	Any host unhealthy, or the check fails twice
Errors	ALB HTTPCode_Target_5XX_Count	5xx rate above normal for 5 minutes
Performance	ALB TargetResponseTime (p95, p99)	p95 above your target
CPU	EC2 / RDS CPUUtilization	Above 80% for 10 to 15 minutes
Memory	Agent mem_used_percent; RDS FreeableMemory	Above 85% used, or falling steadily
Disk	Agent disk_used_percent; RDS FreeStorageSpace	Above 80% used
Database	DatabaseConnections, ReadLatency / WriteLatency, ReplicaLag	Near max connections, or lag growing
Queues	SQS ApproximateAgeOfOldestMessage	Messages waiting too long

ROUTE ALERTS	CloudWatch alarm → SNS topic → email, Slack or PagerDuty.
AVOID NOISE	Alert on symptoms customers feel; use evaluation periods so a spike doesn't page anyone; composite alarms to combine signals.

DASHBOARDS	One CloudWatch dashboard per service: traffic, errors, latency, saturation.
LOGS	Ship app and PHP error logs to CloudWatch Logs; metric filters turn error lines into alarms.

11

YOUR STORY

Typeset: memory on a small server

Your own production system, and exactly the kind of issue they want caught early. Rehearse it.

SITUATION	Typeset, a PDF render API you built and run in production on a server with 960 MB of RAM.
PROBLEM	Large renders ran out of memory and the process was killed (OOM).
WHAT YOU DID	Diagnosed it as memory, not a code bug; added swap as an immediate fix so renders complete.
WHAT'S NEXT	Limit how many renders run at once and add a timeout, so one big job can't starve the rest.
THE LESSON	Monitor memory, not just CPU, and alert before it runs out, not after. That's the point of the role's monitoring work.

> **Make it theirs**
>
> Close with how you'd apply it at Workhorse: *"On AWS I'd install the CloudWatch agent so memory is visible, alarm at 85%, and look for which requests drive the peaks."*

Investigating a production issue

Investigate

1 **Scope it**

Who's affected, since when, what changed? Check recent deploys first.

2 **Gather evidence**

Logs, metrics and the database, around the time it started. Reproduce it if you can.

3 **Mitigate, then fix**

Roll back or work around to stop the harm; then find and fix the root cause.

4 **Write it up**

What happened, why, and what stops it happening again. Add the alarm that would have caught it.

> **Working with the outsourced infrastructure team**
> Escalate with evidence: the time it started, the metrics, the logs, what you've ruled out. It saves them hours and earns trust. Learn from each handover so next time you can fix it yourself.

12

THE PLATFORM

PHP refresher

PHP is a bonus for them, which makes it an advantage for you. Be ready to talk about legacy code too.

Topic	What to say
PHP 8 features	Typed properties, constructor promotion, enums, match, readonly, named arguments, nullsafe ?→.
Strict types	declare(strict_types=1) catches type bugs early.
PDO	Prepared statements with bound parameters, always. Never build SQL from strings.
Composer	Dependencies and PSR-4 autoloading; commit composer.lock.
Testing	PHPUnit or Pest; PHPStan for static analysis.

Topic	What to say
Legacy code	Add tests around it before changing it; refactor in small steps; don't rewrite.

```php
<?php declare(strict_types=1);

final class TicketRepository
{
    public function __construct(private readonly PDO $db) {}

    /** @return list<array{id: int, subject: string}> */
    public function openFor(int $agentId, int $limit = 20):
array
    {
        $stmt = $this->db->prepare(
            'SELECT id, subject FROM tickets
             WHERE status = :status AND assigned_to = :agent
             ORDER BY created_at DESC LIMIT :limit'
        );
        $stmt->bindValue(':status', 'open');
        $stmt->bindValue(':agent', $agentId,
PDO::PARAM_INT);
        $stmt->bindValue(':limit', $limit, PDO::PARAM_INT);
        $stmt->execute();

        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }
}
```

Say this

"I've worked a lot in PHP and Laravel, so I can be useful in the codebase from the first week, and I'm comfortable in older code: I'd put tests around something before I change it."

13

YOUR EXAMPLES

Stories to have ready

Use situation, task, action, result. Two minutes each, with a number in the result where you can.

Question	Your story
Something you built and run in production	Typeset: a render API in production; how you deploy, monitor and fix it.
A production problem you diagnosed	The Typeset OOMs: memory not code, swap now, concurrency and timeouts next.
How you use AI tools	Claude Code daily: planning first, small diffs, reading every line, tests before done.
A time AI got it wrong	Have one specific example: what it invented or broke, and how you caught it.
Improving a system you didn't build	A legacy PHP or Laravel app you modernised without a rewrite.
A slow query you fixed	The query, what EXPLAIN showed, the index, before and after timings.
Automating something repetitive	A script, pipeline or agent workflow that saved you or a team time.
Helping a non-technical colleague	Explaining an issue to a client or support person in plain words.

Tonight

Fill in the two you're least sure of, especially *a time AI got it wrong* and *a slow query you fixed*, with real details. Specifics are what make a story believable.

14

PREPARATION

Questions they’re likely to ask

Technical

- Walk me through how you’d pick up a Jira ticket you don’t fully understand.
- How do you review code an AI tool has written? What do you look for?
- Explain how a RAG pipeline works. How would you improve one that gives poor answers?
- How would you decide when it’s safe to send an AI reply to a customer without a person checking it?
- What is prompt injection, and how would you protect a helpdesk pipeline from it?
- Write a query: customers who raised more than three tickets in the last 30 days.
- A page is slow and you suspect the database. What do you do?
- How do you add a column to a table with 50 million rows without downtime?
- What would you monitor on AWS for a PHP app, and how would you alert on it?
- The site is down. Talk me through your first ten minutes.

About you

- Why this role, and why now?
- Tell me about something you built that you’re proud of.
- How do you balance working independently with keeping the team informed?
- What excites you about AI, and what worries you about it?

15

PREPARATION

Questions to ask them

Pick three or four that fit the conversation. They signal that you've thought about the job.

The helpdesk and RAG pipeline

- What does the RAG pipeline do today, and where does it fall short?
- What does it index, and how do you know if its answers are good?
- How many tickets a month, and which kinds would you most like automated?

Development

- Which AI development tools do you use, and what's your review process for their output?
- What does the path from ticket to production look like? How often do you deploy?
- Where is the codebase hardest to work in?

Operations and the team

- What monitoring is in place on AWS now, and what has gone wrong that it didn't catch?
- How is the work split with the outsourced infrastructure team?
- What would success look like in the first three and six months?

Closing line

"This role covers everything I enjoy: shipping real changes, making AI tools reliable rather than just using them, and keeping a production system healthy. I'm used to owning things end to end, and I'd like to help make the helpdesk automation something the team really trusts."

16

BEFORE THE INTERVIEW

Tonight and tomorrow

Tonight

1 **Look them up**

Workhorse's product, customers and recent news; who's interviewing you.

2 **Rehearse three stories out loud**

Typeset, how you use Claude Code, and a slow query you fixed.

3 **Write three SQL queries by hand**

A join, a GROUP BY with HAVING, and a window function.

4 **Explain RAG in two minutes**

Out loud, without notes: index, retrieve, rerank, generate, evaluate.

Tomorrow

5 **Re-read the job description**

Its words are your vocabulary: *well-targeted, maintainable, secure, properly tested*.

6 **Check your setup**

Camera, sound, a working editor and terminal in case of a live exercise.

7 **Have this guide open**

Plus your questions for them, and a glass of water.